A Comparison of Neural Recommender System Architectures

Ken Low
University of Colorado, Boulder

ABSTRACT

Recommender systems use user-item interactions and related features to estimate the relevance of candidate items. This study compares six neural recommender architectures, three hyperparameter-search methods, and four optimizer families under a shared, intentionally short training protocol. Five-seed full-data experiments were run on MovieLens 100K and ogbl-collab using an NVIDIA RTX PRO 6000 GPU. In the model-family comparison, the DNN had the highest MovieLens test AUC (0.4627 +/- 0.0096), while PSL-DNN led ogbl-collab (0.9477 +/- 0.0015). Among hyperparameter search methods, Grid search ranked first on MovieLens (0.6562 +/- 0.0183), whereas Differential Evolution-style search ranked first on ogbl-collab (0.9178 +/- 0.0051). When comparing optimizers, PSO achieved the best result on MovieLens (0.5107 +/- 0.0288), and Adam led ogbl-collab (0.8650 +/- 0.0243). The results are descriptive rather than formal significance tests and show that architecture, tuning, and optimization effects depend strongly on the dataset and protocol.

1 INTRODUCTION

Recommender systems are widely used in many applications such as movie recommendations to users on entertainment platforms, featured products on online commerce sites, and even friend suggestions on social networks. To make effective recommendations that align with users' preferences, a recommender system must learn about the characteristics of different users and items and then infer which items to match with a specific user. Machine learning in recommender systems is well suited to graph-based representations because data on users and items can be mapped naturally to a graph structure, and graphs can capture complex, nonlinear relationships (Wang et al., 2021). In addition, neural network modeling has become more popular because of its expressive power and its ability to capture more complex relationships, which makes neural methods a natural choice for recommender-system development (Gridach, 2020).

2 RELATED WORK

Neural recommender systems have been widely studied as a way to model user-item interactions with learned distributed representations rather than manually engineered similarity rules alone. Embedding-based deep neural recommenders treat users and items as trainable vectors and then score candidate interactions with feedforward networks, as in neural collaborative filtering approaches such as He et al. (2017). This line of work motivates the non-graph Deep Neural Network (DNN) baseline in the present study, where recommendation is framed as an edge-prediction problem over learned node embeddings.

Graph neural networks extend this idea by allowing node representations to absorb information from neighboring nodes and edges. In recommender settings, graph-based learning has been used to model user-item interactions more explicitly than plain embedding methods, especially when interactions can be represented as edges in graphs, as shown by neural graph collaborative filtering and subsequent simplified graph recommenders (Wang et al., 2019; He et al., 2020). This literature motivates the inclusion of a graph neural network baseline in the study and provides the basis for comparing graph-aware and non-graph neural recommenders under a shared experimental protocol.

A second relevant body of work concerns hybrid recommender architectures that introduce additional inductive biases into neural models. Sequence-aware recommenders use recurrent networks such as gated recurrent units (GRUs) and long short-term memory (LSTM) models to capture temporal order in user behavior, as in session-based recommendation models such as Hidasi et al. (2016), while graph-and-sequence hybrids combine structural information from interaction graphs with temporal summaries of past interactions. Related efforts have also explored combining neural recommenders with hidden-state or regime-switching ideas, which motivates the RHMM-inspired variant examined in this study (Djellali & Adda, 2020).

Another important thread in the literature augments neural recommendation with relational rules or symbolic structure. Probabilistic Soft Logic (PSL) and related neuro-symbolic approaches show that handcrafted relational signals, such as shared neighbors or degree-based patterns, can complement learned latent representations (Bach et al., 2017). The PSL-DNN model in the present study follows this general direction, although it should be understood

as a lightweight PSL-style approximation based on graph heuristics rather than a full probabilistic logic inference system.

Reinforcement learning (RL) has also influenced recommender-system research, especially in settings where a model must adapt decisions over time or choose among competing strategies. In addition to full RL formulations for recommendation, lighter policy-based mechanisms have been used to control components of neural architectures (Zou et al., 2019). This body of work motivates the RL-guided aggregation variant in the current study, where a bandit-style policy selects among alternative graph-aggregation strategies during training.

Beyond model architecture, prior work has shown that hyperparameter selection and optimization strategy can materially affect neural-network performance. Grid search and random search remain standard baselines for hyperparameter tuning, with Bergstra and Bengio (2012) showing the practical strength of random search, while population-based search procedures such as Differential Evolution provide an alternative for exploring mixed continuous and discrete search spaces (Storn & Price, 1997). Similarly, gradient-based optimizers such as stochastic gradient descent and Adam are central to modern deep learning (Kingma & Ba, 2014), whereas population-based training methods such as particle swarm optimization and evolutionary strategies provide gradient-free alternatives (Kennedy & Eberhart, 1995). These strands of research motivate the study's additional comparisons of hyperparameter-search methods and optimizer families.

Taken together, the prior literature suggests that recommender performance depends not only on whether a model is neural, graph-based, sequential, symbolic, or policy-guided, but also on how the model is tuned and optimized. Much of this prior work, however, evaluates these issues in isolation. The present study addresses that gap by comparing model families, hyperparameter-search strategies, and optimizer classes within a single experimental harness on recommendation-style edge-prediction tasks.

3 METHODOLOGY

The goal of the experiment is to compare neural recommender variants under a shared experimental harness. Each comparison is repeated with seeds 42, 123, 456, 789, and 2026. Predictive performance is summarized as the mean and sample standard deviation across the five independently trained trials, alongside wall-clock time and resource usage.

3.1 Datasets

The MovieLens 100K dataset is a user-item rating table and serves as the smaller implicit-feedback benchmark. After retaining ratings of at least 4, the full chronological split contains 44,300 training, 5,537 validation, and 5,538 test positive edges. The ogbl-collab dataset is a large author-collaboration graph; its full official OGB split contains 1,179,052 training, 60,084 validation, and 46,329 test positive edges. Together, the datasets contrast bipartite recommendation with a larger homogeneous graph-learning proxy.

3.2 Data Transformation

The datasets are converted into edge-prediction tasks. MovieLens interactions are ordered by timestamp and split 80/10/10 so that training precedes validation and test interactions. ogbl-collab retains the official OGB link-prediction split. Positive edges come from observed interactions, and negative edges are sampled from unobserved node pairs; an unobserved pair is therefore a sampled evaluation negative, not a verified dislike.

In MovieLens 100K, ratings of 4 and 5 are treated as positive interactions because they provide a conservative signal of preference. Ratings below 4 are excluded rather than labeled as dislikes, avoiding the stronger and less defensible assumption that every lower or neutral rating is negative. The resulting task is implicit-feedback link prediction rather than explicit rating regression.

3.3 Model Comparisons

The recommender models were compared along three axes: model family, hyperparameter-search strategy, and optimization method. Every compared method uses the same five trial seeds and dataset-specific training schedule.

3.3.1 Model-Family Comparison

The first suite of experiments compared different neural recommender architectures. Each model family was trained under the same default hyperparameters and training-budget settings so that the primary source of variation was architectural rather than procedural. The comparison begins with two baselines: a non-graph deep neural network (DNN) that scores candidate node pairs from learned embeddings, and a graph neural network (GNN) that augments those embeddings with neighborhood information from the training graph.

3.3.2 Deep Neural Network (DNN)

The non-graph baseline recommender is implemented with PyTorch's EmbeddingMLPRecommender class. For each mini-batch, the model assigns an embedding to each entity such as a user or item, constructs pairwise features from the source and destination embeddings, and passes those features to a multilayer perceptron (MLP) scoring head. The MLP scoring head consists of the following components:

- a linear layer
- the rectified linear unit (ReLU) function
- the Dropout regularization method, which randomly turns off a fraction of layer activations during training only
- followed by the final linear layer that gives the output logit score

In this study, the DNN scoring head uses only two linear layers because network depth is not the main variable of interest and a shallow architecture keeps the baseline computationally simple. The predicted logit for each sampled edge is compared with its binary label, and backpropagation updates both the embedding table and the scoring-head weights to reduce binary cross-entropy loss.

3.3.3 Graph Neural Network (GNN)

The graph baseline recommender is implemented with PyTorch's GNNRecommender class. Each graph node's embedding is updated by aggregating one-hop neighbor information over the training graph using mean message aggregation. The resulting graph-aware node representations are then combined into pairwise features and passed to the same MLP scoring head used in the DNN baseline. The graph neural network consists of the following layers and steps:

- Two GraphEncoder layers, each of which applies:
  - A linear layer to update the graph node's current representation
  - Another linear layer to update the aggregated neighbor representation
  - ReLU
  - Dropout for regularization
- A fixed combination of the initial embeddings and graph-layer outputs (mean in the baseline GNN)
- Computing feature vectors representing each pair of graph nodes in the training sample
- An MLP classifier to predict the presence of an edge for each pair, consisting of:
  - A linear layer
  - ReLU
  - Dropout for regularization
  - Another linear layer that outputs the logit for whether the edge should exist

The predicted logits are compared with sampled labels using binary cross-entropy loss, and the optimizer updates the trainable parameters to minimize that loss.

3.3.4 Hybrid Hidden Markov-Neural Network

This hybrid neural network is inspired by RHMM-style sequential recommendation (Djellali & Adda, 2020), but it is not a full hidden Markov model implementation. For each candidate pair of nodes, the model extracts the source node's recent interaction history in temporal order, encodes that sequence with a single-layer gated recurrent unit (GRU), and then infers a soft latent regime mixture from the GRU state – a probability-weighted mixture of several possible hidden behavioral states of the source node. The GRU summary, regime vector, and destination-node embedding are concatenated and scored with the same MLP head used elsewhere. The significance is that the temporal order of previous interactions is explicitly modeled instead of being ignored.

3.3.5 Hybrid Probabilistic Soft Logic (PSL) Deep Neural Network

This hybrid model augments the embedding-based DNN with PSL-like relational features. The implementation appends handcrafted graph statistics such as source and destination degree, the degree-product term, and neighbor-overlap ratios to the learned pairwise embedding features, then scores the combined vector with the MLP classifier. The model is therefore PSL-like in the sense that it injects soft relational evidence, not because it runs an external PSL solver.

3.3.6 Hybrid Bidirectional Long Short-Term Memory (LSTM) Graph Neural Network

This graph-neural variant uses a bidirectional LSTM to summarize each source node's recent sequence of historical interactions, then combines that sequence summary with graph-aware source and destination embeddings produced by the GNN encoder. The resulting feature vector is scored by the MLP head to estimate whether the candidate edge should exist.

3.3.7 Hybrid RL-Optimized GNN Aggregation

This variant keeps the same GNN encoder and MLP scorer but adds a lightweight bandit-style policy over several ways of combining graph-layer outputs (last, mean, and a residual-style weighted combination). During training, the policy samples an aggregation expert, receives reward based on the batch loss, and updates its preference logits. At evaluation time, the expert with the strongest learned policy preference is used.

3.4 Comparison of Hyperparameter Search Methods

The second suite compares grid search, random search, and Differential Evolution (DE)-style search. Each method receives 24 candidate configurations over embedding size, hidden-layer size, dropout, and learning rate. Every candidate is fit once per seed and ranked only by mean validation AUC. The highest-ranked configuration for each method is then retrained once per seed and evaluated on the test split; validation-search rows contain no test metrics. This protocol prevents the test set from influencing hyperparameter selection.

Grid search evaluates every combination in a predefined discrete hyperparameter grid. Random search samples hyperparameter combinations independently from the predefined search space, providing a less exhaustive but cheaper alternative to grid search. In DE-style search, a small population of candidate hyperparameter vectors is initialized, evaluated, and then iteratively mutated and crossed over using differences between other candidates in the population. A trial candidate replaces the incumbent when it improves validation area under the curve (AUC). The search runs for a fixed number of generations rather than until a guaranteed optimum is found.

3.5 Optimizer Comparison: Gradient-Based vs. Population-Based Training

The third suite of experiments compared gradient-based optimizers with population-based alternatives. Adam and stochastic gradient descent (with momentum 0.9) were the gradient methods, while particle swarm optimization (PSO) and an evolution-strategy-style optimizer were the population-based methods. The optimizer comparison used a dataset-appropriate baseline in each case: a DNN on MovieLens 100K and a GNN on ogbl-collab.

Adam is one of the most frequently cited optimization algorithms for neural networks, adapting the learning rate with estimates of the first and second moments of the gradients (Kingma & Ba, 2014). SGD computes the gradient from a mini-batch of pairs of graph nodes. In PSO, the model generates multiple sets of parameters and represents each set as a particle; personal and global best values guide updates until the particles converge. Evolutionary strategy is a population-based gradient-free optimizer that iteratively perturbs a parameter vector and shifts it toward perturbations that reduce loss.

3.6 Evaluation Protocol and Compute Environment

The experiments were run once in a quicker mode on a laptop, where only a portion of each dataset's training sample was used at the training step, then again in full in a University of Colorado's Research Computing (CURC) GPU environment, where the entire training sample was used for training. The full-mode experiment preserves the intentionally short training schedules used by the quick baseline; full mode changes dataset sampling, not epoch or mini-batch budgets. Gradient runs use balanced batches of 512 positive and 512 sampled negative edges. Depending on the suite and dataset, they perform 24-175 optimizer updates. Population-based optimizer runs evaluate a fixed balanced batch of 256 positive and 256 negative edges. The complete run used Python 3.11.13, PyTorch 2.11.0+cu130, CUDA 13.0, eight allocated CPU cores, 64 GB RAM, and one NVIDIA RTX PRO 6000 Blackwell Server Edition GPU with compute capability 12.0.

4 RESULTS

Tables 1-3 report the completed full-mode run. Test AUC, test average precision (AP), and binary accuracy are shown as mean (sample SD) across five seeds. Runtime is mean per final test fit; RAM and GPU columns are the mean of the per-trial peaks. HPO candidate-search time is totaled separately because equal candidate counts do not guarantee equal runtime.

4.1 Model-Family Comparison

Dataset	Model	Method	Test AUC	Test AP	Accuracy	Mean Time (s)	Mean RAM (MB)	Mean GPU (MB)
movielens_100k	dnn	adam	0.4627 (0.0096)	0.4869 (0.0052)	0.4733 (0.0034)	0.128	2384.5	22.9
movielens_100k	psl_dnn	adam	0.3932 (0.0247)	0.4569 (0.0087)	0.4689 (0.0026)	1.064	2399.0	23.0
movielens_100k	gnn	adam	0.3708 (0.0116)	0.4460 (0.0026)	0.4677 (0.0067)	0.242	2384.7	49.5
movielens_100k	gnn_bilstm	adam	0.3638 (0.0278)	0.4457 (0.0086)	0.4620 (0.0027)	0.911	2399.2	131.3
movielens_100k	rl_gnn	bandit	0.3584 (0.0557)	0.4426 (0.0231)	0.4472 (0.0420)	0.276	2401.0	49.4
movielens_100k	rhmm	adam	0.3541 (0.0288)	0.4437 (0.0092)	0.4636 (0.0109)	0.948	2398.9	124.5
ogbl_collab	psl_dnn	adam	0.9477 (0.0015)	0.9168 (0.0017)	0.8976 (0.0014)	1.880	2483.6	349.8
ogbl_collab	dnn	adam	0.9192 (0.0036)	0.8540 (0.0095)	0.8562 (0.0057)	0.166	2483.6	349.8
ogbl_collab	rl_gnn	bandit	0.9013 (0.0166)	0.7998 (0.0488)	0.8312 (0.0240)	1.453	2483.6	1224.3
ogbl_collab	gnn	adam	0.8989 (0.0064)	0.7808 (0.0189)	0.8138 (0.0085)	0.783	2483.6	1221.3
ogbl_collab	rhmm	adam	0.8350 (0.0020)	0.6592 (0.0055)	0.7623 (0.0141)	1.604	2483.6	425.1
ogbl_collab	gnn_bilstm	adam	0.8340 (0.0024)	0.6580 (0.0044)	0.7846 (0.0148)	2.628	2483.6	1221.5

Table 1. Full-mode model-family results; predictive metrics are mean (sample SD) over five seeds.

The model-family results are strongly dataset-dependent. On MovieLens, DNN ranks first among the default architectures, but its AUC of 0.4627 and every other family result are below the 0.5 chance-ranking reference. The default-family MovieLens results therefore do not establish useful discrimination and should not be interpreted as evidence of practical recommendation quality. On ogbl-collab, PSL-DNN ranks first at 0.9477, followed by DNN at 0.9192. The graph-sequential and recurrent hybrids required more time or GPU memory without exceeding these simpler baselines under the short schedule.

4.2 Hyperparameter-Search Comparison

Dataset	Search	Test AUC	Test AP	Accuracy	Final Time (s)	Search Time (s)	Peak RAM (MB)	Peak GPU (MB)	Selected Config
movielens_100k	grid	0.6562 (0.0183)	0.6311 (0.0174)	0.6098 (0.0177)	0.066	7.6	2318.3	25.0	emb=16; hid=96; drop=.35; lr=.000631
movielens_100k	DE-style	0.6542 (0.0108)	0.6215 (0.0088)	0.5883 (0.0114)	0.066	7.6	2318.4	25.0	emb=16; hid=96; drop=.296; lr=.00145
movielens_100k	random	0.6528 (0.0145)	0.6214 (0.0121)	0.5913 (0.0172)	0.066	7.6	2318.3	25.0	emb=16; hid=96; drop=.049; lr=.00110
ogbl_collab	DE-style	0.9178 (0.0051)	0.8438 (0.0185)	0.8491 (0.0096)	0.880	64.6	2483.5	1743.9	emb=48; hid=96; drop=.047; lr=.01
ogbl_collab	grid	0.9114 (0.0078)	0.8328 (0.0156)	0.8418 (0.0096)	0.875	48.8	2521.3	1743.9	emb=48; hid=96; drop=0; lr=.01
ogbl_collab	random	0.9047 (0.0121)	0.8058 (0.0275)	0.8334 (0.0174)	0.880	51.9	2477.5	1743.9	emb=48; hid=64; drop=.130; lr=.00817

Table 2. Full-mode HPO results; final test metrics are mean (sample SD), with total validation-search time over 120 candidate fits per method.

No search method is uniformly best. On MovieLens, grid search has the highest mean test AUC (0.6562), narrowly ahead of DE-style (0.6542) and random search (0.6528). On ogbl-collab, DE-style search ranks first (0.9178), followed by grid (0.9114) and random search (0.9047). Grid has the lowest total ogbl-collab candidate-search time at 48.8 seconds, compared with 51.9 seconds for random search and 64.6 seconds for DE-style search.

4.3 Optimizer Comparison

Dataset	Model	Optimizer	Test AUC	Test AP	Accuracy	Mean Time (s)	Mean RAM (MB)	Mean GPU (MB)
movielens_100k	dnn	pso	0.5107 (0.0288)	0.5092 (0.0269)	0.5092 (0.0182)	0.146	2317.0	20.6
movielens_100k	dnn	sgd	0.4986 (0.0276)	0.5035 (0.0135)	0.4984 (0.0031)	0.203	2309.2	20.8
movielens_100k	dnn	evolutionary	0.4971 (0.0276)	0.5047 (0.0141)	0.5011 (0.0029)	2.080	2318.0	20.6
movielens_100k	dnn	adam	0.4511 (0.0134)	0.4843 (0.0039)	0.4780 (0.0023)	1.153	2304.2	21.0
ogbl_collab	gnn	adam	0.8650 (0.0243)	0.7251 (0.0515)	0.7790 (0.0278)	0.553	2512.9	699.7
ogbl_collab	gnn	pso	0.5416 (0.0464)	0.3379 (0.0411)	0.4661 (0.0875)	4.835	3997.5	340.2
ogbl_collab	gnn	sgd	0.5210 (0.0643)	0.3063 (0.0313)	0.4838 (0.1870)	0.521	2483.9	684.8
ogbl_collab	gnn	evolutionary	0.5032 (0.0533)	0.2960 (0.0243)	0.4699 (0.1954)	7.264	3614.2	340.2

Table 3. Full-mode optimizer results; predictive metrics are mean (sample SD) over five seeds.

On MovieLens, PSO has the highest mean test AUC (0.5107), followed by SGD (0.4986), evolutionary training (0.4971), and Adam (0.4511), although the standard deviations are large relative to several mean differences. On ogbl-collab, Adam is clearly strongest at 0.8650, while PSO, SGD, and evolutionary training range from 0.5032 to 0.5416. The population-based methods also have the longest mean runtimes on ogbl-collab. The optimizer results therefore do not support a universally superior optimizer family.

Overall, the full-mode runs favor a simple DNN or explicit PSL-style relational features in the architecture comparison, but the best search and optimizer choices vary by dataset. Hyperparameter tuning substantially improves the MovieLens DNN relative to its default family configuration, emphasizing that conclusions about architecture should remain conditional on the short training and tuning protocol.

5 CONCLUSION

Within the bounds of this lightweight full-data harness, added architectural complexity does not consistently improve recommendation-style edge prediction. DNN leads the default MovieLens family comparison, although all default MovieLens family AUCs are below 0.5, and PSL-DNN leads ogbl-collab. Grid and DE-style search divide the two HPO benchmarks, while PSO leads the MovieLens optimizer comparison and Adam leads ogbl-collab. These five-seed descriptive results show that dataset, feature design, tuning, and optimizer choice materially affect rankings; they do not justify a universal winner or claims of statistical significance.

6 FUTURE WORK

Several directions remain. First, future experiments should test longer training schedules, stronger baselines, and nested or repeated resampling. Second, paired uncertainty estimates and prespecified statistical comparisons would clarify which five-seed differences are robust. Third, ranking metrics such as NDCG@K, Recall@K, and Hit Rate@K should supplement sampled edge-prediction metrics. Finally, the hybrid variants should be implemented more faithfully and evaluated on additional recommendation datasets and production-like temporal protocols.

REFERENCES

Bach, S. H., Broecheler, M., Huang, B., & Getoor, L. (2017). Hinge-loss Markov random fields and probabilistic soft logic. Journal of Machine Learning Research, 18(109), 1–67. https://jmlr.org/papers/v18/15-631.html

Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. Journal of Machine Learning Research, 13, 281–305. https://www.jmlr.org/papers/v13/bergstra12a.html

Djellali, C., & Adda, M. (2020). A new hybrid deep learning model based-recommender system using artificial neural network and hidden Markov model. Procedia Computer Science, 175, 214–220. https://doi.org/10.1016/j.procs.2020.07.032

Gridach, M. (2020). Hybrid deep neural networks for recommender systems. Neurocomputing, 413, 23–30. https://doi.org/10.1016/j.neucom.2020.06.025

He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T.-S. (2017). Neural collaborative filtering. In Proceedings of the 26th International Conference on World Wide Web (pp. 173–182). https://doi.org/10.1145/3038912.3052569

He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., & Wang, M. (2020). LightGCN: Simplifying and powering graph convolution network for recommendation. In Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (pp. 639–648). https://doi.org/10.1145/3397271.3401063

Hidasi, B., Karatzoglou, A., Baltrunas, L., & Tikk, D. (2016). Session-based recommendations with recurrent neural networks. arXiv. https://doi.org/10.48550/arXiv.1511.06939

Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. In Proceedings of ICNN'95 - International Conference on Neural Networks (pp. 1942–1948). https://doi.org/10.1109/ICNN.1995.488968

Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv. https://doi.org/10.48550/arXiv.1412.6980

Storn, R., & Price, K. (1997). Differential evolution: A simple and efficient heuristic for global optimization over continuous spaces. Journal of Global Optimization, 11(4), 341–359. https://doi.org/10.1023/A:1008202821328

Wang, X., He, X., Wang, M., Feng, F., & Chua, T.-S. (2019). Neural graph collaborative filtering. In Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval (pp. 165–174). https://doi.org/10.1145/3331184.3331267

Wang, S., Hu, L., Wang, Y., He, X., Sheng, Q. Z., Orgun, M. A., Cao, L., Ricci, F., & Yu, P. S. (2021). Graph learning based recommender systems: A review. arXiv. https://doi.org/10.48550/arXiv.2105.06339

Zou, L., Xia, L., Ding, Z., Song, J., Liu, W., & Yin, D. (2019). Reinforcement learning to optimize long-term user engagement in recommender systems. In Proceedings of the 25th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (pp. 2810–2818). https://doi.org/10.1145/3292500.3330668